

Project Nature & Expectations

1. Each project must include:

- A clearly defined attack model
- A clearly defined prevention / mitigation mechanism
- Mathematical justification of:
 - Why the attack works
 - Why the prevention improves security
- Without mathematical reasoning, the project will not be evaluated, even if code works

2. Programming Language

- Python 3.10 or above only
- Hard-coded constants are allowed (AES S-box, known MD5 collisions, NIST vectors)

3. Implementation Requirement

- Implementing from scratch means writing the algorithm logic using basic Python constructs (loops, conditionals, arithmetic) without calling cryptographic libraries. Use of built-in arithmetic (int, pow(a, b, mod)), hard-coded constants, reduced rounds, and toy parameters is allowed. Industrial-grade optimization is NOT expected.
- Core cryptographic algorithms must be implemented from scratch
- Reduced rounds (AES/DES) are allowed only with explanation

Example 1: RSA (From Scratch)

Generate p and q using Miller-Rabin

Compute n, $\phi(n)$

Compute d using Extended Euclidean Algorithm

Encrypt/decrypt using pow()

from Crypto.PublicKey import RSA (Packages are not allowed to use)

Example 2: AES (From Scratch)

Write SubBytes using a lookup table

Implement ShiftRows and MixColumns

Use reduced rounds (4–6)

from Crypto.Cipher import AES (Packages are not allowed to use)

4. Attack Success Requirement

- A test case refers to a single automated execution of the same attack under different inputs (keys, messages, parameters, or random values). Not expected to design multiple attacks. Test cases should be generated programmatically, and success rate should be computed automatically.
- Attack must succeed in $\geq 90\%$ of 20–25 test cases
- Success rate = (successful attacks / total test cases) $\times 100$

5. Prevention Requirement

- Prevention must work on practical key sizes (RSA/DH: 1024–2048 bits, AES: full key size)

6. Live Demonstration

- Mandatory live demo during reviews
- Static screenshots or prerecorded videos are not accepted

7. GUI Requirement

- Simple Tkinter GUI with exactly:
 - Generate Keys / Parameters
 - Run Attack
 - Apply Prevention
 - Show Graphs
 - Must include text log + red (vulnerable) / green (secure) indicators

8. Mandatory Graphs (Minimum 4)

Every project must include at least these four graphs:

- Before vs After attack success rate (bar chart)
- Time vs key / parameter size (line chart)
- Confidentiality / Integrity rate comparison (bar chart)
- Attack vs Prevention latency overhead (bar chart)

9. AI Tool Usage Policy

- AI tools (ChatGPT, Copilot, etc.) may be used only for learning support
 - Concept clarification
 - Syntax reference
- Plagiarism check on code + report, following awarded:
 - 0–30% → 3 marks
 - 31–65% → 2 marks
 - > 66% → 1 mark

10. Presentation Rules

- PowerPoint presentation is mandatory for both reviews
- Every technical claim must be supported by:
 - Mathematical reasoning
 - Graphs or results
 - Academic references
- All team members must present
- No late presentations
- Review absence = ZERO marks
- Individual contribution will be verified

Review Evaluation

Review–1 (15 Marks)

Dates: February 25 – February 27

Mandatory:

- Title finalization
- Problem statement
- Literature survey
- Mathematical background
- System & attack model
- Partial attack results
- PPT

Marks:

- Project clarity – 5
- Mathematical understanding – 4
- Attack model & design – 4
- Presentation – 2

Review–2 (Final Review) (15 Marks)

Dates: March 25 – March 27

Mandatory:

- Attack implementation
- Prevention mechanism
- Mathematical justification
- Graphs & comparison
- Final report (25–30 pages)
- PPT + Live demo

Marks:

- Implementation correctness – 5
- Security improvement & math – 4
- Results, graphs & demo – 3
- Documentation quality – 3